Design and Realization of Mixed-Technology Systems

Mirco Meiners Tim Ziemann

2026-11-05

Table of contents

1. Unsyllabus **1**

I. Lecture **3**

2. Introduction and Survey **5**
2.1. Course Objectives . . . 5
2.2. Scientific Computing / Agentic Engineering . . . 5
2.3. EDA Tools . . . 5
2.4. Operating System Tools . . . 6
2.5. Code Editors . . . 6
2.6. Scientific Writing . . . 6
2.7. Design of a General-Purpose Second-Order Switched-Capacitor $\Delta\Sigma$-ADC . . . 7
2.8. Course Prerequisites . . . 7

II. Tutorials **9**

3. Analog Inverter Tutorial **11**

4. Introduction **13**

5. Full design flow using IIC-OSIC tools **15**
5.1. Schematic entry using Xschem . . . 15
5.2. Simulation using NgSpice . . . 16
5.3. Output Results . . . 20
5.4. Post processing and simulation using python . . . 21
5.5. Introduction to Layout . . . 23
5.5.1. Designing Layout using Klayout . . . 23

6. Klayout-PEX (KPEX) **29**
6.1. Why Do We Need PEX? . . . 29
6.2. PEX using Klayout-PEX tool . . . 29
6.3. Running the `KPEX/MAGIC` Engine . . . 30
6.3.1. Example Command . . . 30
6.4. Post layout simulation. . . . 34
6.5. Results . . . 35

7. Xschem Commands **37**

8. ngspice Commands **39**
8.1. Commands . . . 39

8.2. Options 40
8.3. Convergence Helper 40

Bibliography **41**

1. Unsyllabus

Name	Description
Course	M 2.10 Design and Realization of Mixed-Technology Systems (DRMS)
Term	Winter
Instructor	Prof. Dr.-Ing. M. Meiners, Dipl.-Ing. (FH) T. Ziemann

(c) 2024-2026 Mirco Meiners and co-authors, Hochschule Bremen - City University of Aplied Sciences (HSB), Bremen, Germany

The course makes heavy use of circuit simulation, using **Xschem** for schematic entry and **ngspice** for simulation. The 130nm CMOS technology **SG13G2** from IHP Microelectronics is used.

Tools and PDK are integrated in the **IIC-OSIC-TOOLS** Docker image, which will be used during the coursework.

All course material is made publicly available and shared under the Apache-2.0 license.

Part I.

Lecture

2. Introduction and Survey

2.1. Course Objectives

- Interfacing Microsystems
  - Mixed-technology systems (System-on-Chip, SoC)
  - System analysis
  - System specification
- Design Methodology
  - Seamlessly modeling and design over all physical domains
- PCB (System) and IC Design
  - Architecture
  - Partitioning
  - Layout

2.2. Scientific Computing / Agentic Engineering

- Python (3.13, 3.14)
- Matlab Campus License

2.3. EDA Tools

- PCB / System Design
  - LTspice
  - KiCad EDA
  - Altium Designer
  - SiemensEDA PCB tools
  - cadence System Design & Analysis
- IC / Silicon Design
  - IIC-OSIC-TOOLS (open-source)
  - SiemensEDA IC tools
  - cadence IC Design & Verification
  - synopsys silicon design (IC)

2.4. Operating System Tools

- SSH (Secure Shell)
- Microsoft-Terminal
- Microsoft-PowerShell
- MacOS-Terminal
- Linux/MacOS zsh-tools
- Linux/MacOS bash-it
- Git (Versionskontrolle)
- TortoiseGit – Windows Shell Interface to Git

2.5. Code Editors

- Visual Studio Code
- Spyder IDE
- Thonny (Micro-)Python IDE
- Emacs
- Vim

2.6. Scientific Writing

- quarto
  - Manuscripts
- LaTeX
  - MikTeX (Windows, MacOS, Linux)
  - MacTeX (MacOS)
  - TeXLive (Linux)
- LaTeX Editors
  - TeXStudio
  - TeXMaker
  - TeXWorks
- Collaborative Frameworks
  - ShareLaTeX, Online LaTeX
  - CoCalc - Online LaTeX
    - a service of AcademicCloud
- Literature Management
  - Zotero
  - Citavi
- Data Science

  - Command-line tools
  - Comma-Separated-Values (CSV), Tab-Separated-Values (TSV)
    * csvkit
    * miller
  - Spreadsheet (.xlsx, .ods)

2.7. Design of a General-Purpose Second-Order Switched-Capacitor $\Delta\Sigma$-ADC

Parameter	Symbol	Value	Units
Bandwidth	f_B	~1	kHz
Sampling Frequency	f_S	1	MHz
Signal-to-Noise Ratio	SNR	100	dB
Supply Voltage	VDD	1.5	V

- System level, behavioural model
  - Matlab/Simulink,
  - Python
- IC level, SPICE with behavioural blocks, e.g. OTA and comparator
  - IIC-OSIC-TOOLS IHP130-based analog and digital chip design
- **Final Oral Exam/Project Presentation**

2.8. Course Prerequisites

- Fundamentals of linux operating systems
- Fundamentals of microelectronics
  - Device physics and models
  - Transistor level analog circuits, elementary gain stages
- Fundamentals of analog circuit design
  - Operational amplifier
  - Active filter design
  - Noise analysis
  - Switched-capacitor techniques
- Prior exposure to SPICE, Matlab, Python or equivalent.

Part II.

Tutorials

3. Analog Inverter Tutorial

In … general …

4. Introduction

This tutorial provides a step-by-step introduction to designing and simulating a **simple analog inverter** using the **open-source tools** hosted on the IIC-OSIC platform, specifically with the IHP SG13G2 PDK. The analog inverter, though simple, is a crucial element in analog and mixed-signal circuit design, offering valuable insights into device behavior, biasing, and small-signal performance. The goal of this tutorial is to prrovide a concise overview of the steps needed to create and verify simple CMOS circuits and systems.

5. Full design flow using IIC-OSIC tools

5.1. Schematic entry using Xschem

Schematic entry is performed using Xschem, a powerful open-source schematic editor well-suited for analog and mixed-signal circuit design. In this step, the analog inverter circuit is constructed by placing and connecting NMOS and PMOS transistors from the **IHP SG13G2 PDK** library. Xschem provides an intuitive interface for defining circuit topology, assigning device parameters like W/L ratios, and labeling nodes for simulation. Proper hierarchy and net labeling ensure compatibility with simulation and layout tools used later in the design flow.

1. Start terminal and run IIC-OSIC docker image
2. To select the PDK, use command `iic-pdk ihp-sg13g` refer Figure 5.14

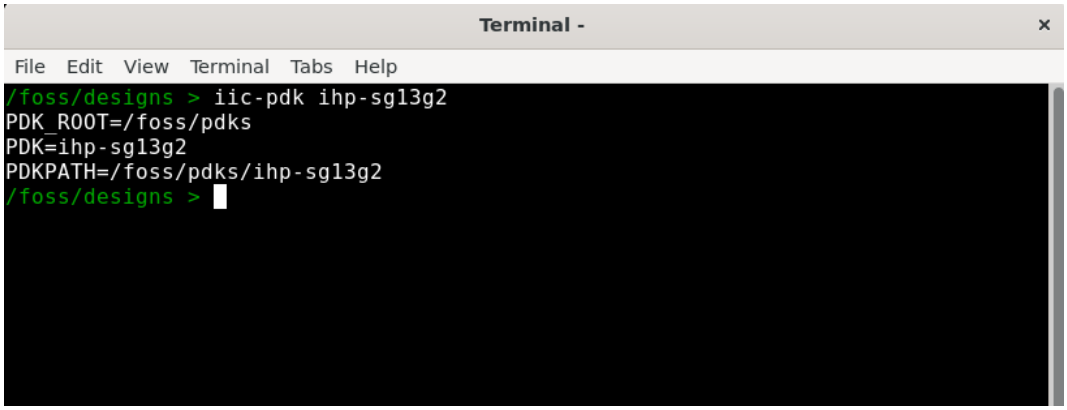


Figure 5.1.: Xschem Entry.

3. Start xschem and create a new schematic. Name it as (Analog_Inverter.sch)
4. Using `Ctrl+i` insert symbol from the sg13g2 library for lv_nmos and lv_pmos where lv stands for low-voltage
5. customize the transistor's dimensions (select the symbol, right click on it and edit its properties) refer Figure 5.2
6. Place an input pin `ipin.sym` and an output pin `opin.sym` the symbols are available in the library `~/foss/tools/xschem/share/xschem/xschem_library/devices` label the name of the input and output pins (select the pin, right click on it and edit its properties)
7. To wire to components, `Ctrl+w`

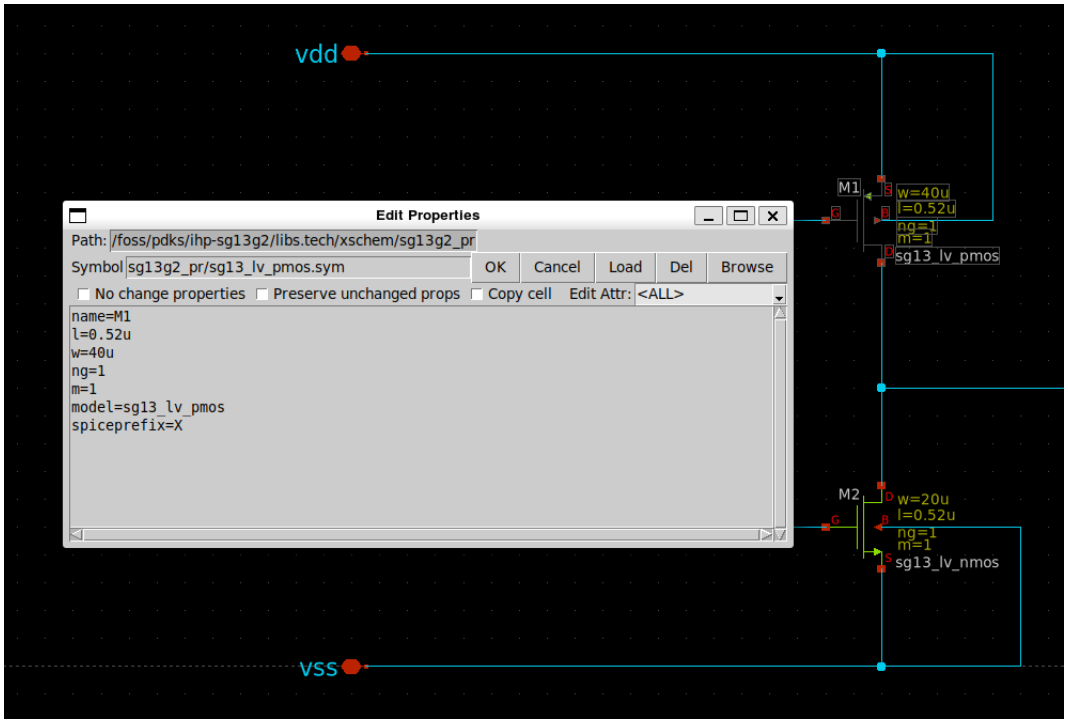


Figure 5.2.: Xschem Entry.

8. Once finished, save the schematic `Ctrl+Shift+S` See Figure 5.4

9. For creating a symbol, `Symbol --> Make symbol from schematic | Ctrl+L`. We can also edit the symbol by opening it in a new tab, see Figure 5.4

10. Now that we have created a symbol, we need to perform DC and Transient analysis using NgSpice.

5.2. Simulation using NgSpice

To perform various analysis, we need to create simulation testbenches.

1. Open new tab in xschem `Ctrl+o`, insert the symbol for inverter (Analog_Inverter.sym) `Ctrl+i`. Design a testbench in xschem see Figure 5.5.

2. The spice directives **MODELS1** and **NGSPICE1** are code.sym instances with the attributes shown in Figure 5.6 and Figure 5.7

3. Click the **Netlist** button to create a netlist which will give us a `.spice` file and then click on **Simulate** button to run the simulation.

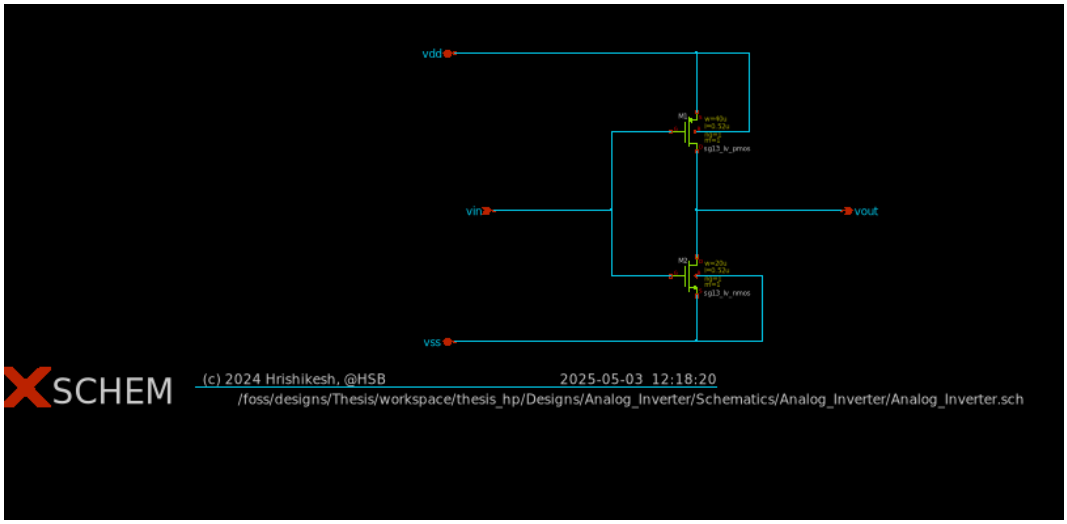


Figure 5.3.: Xschem Entry.

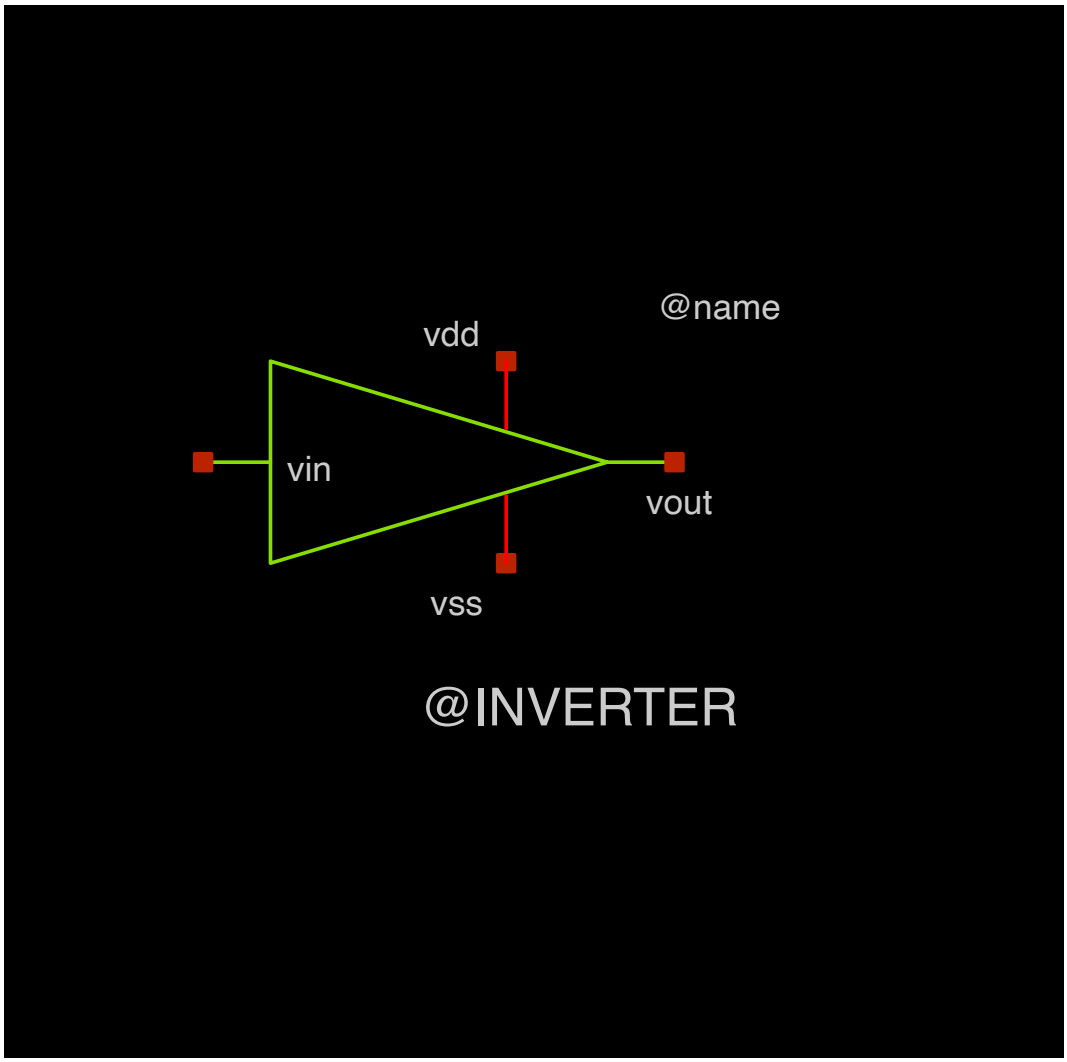


Figure 5.4.: Xschem Symbol.



Figure 5.5.: Xschem Testbench.

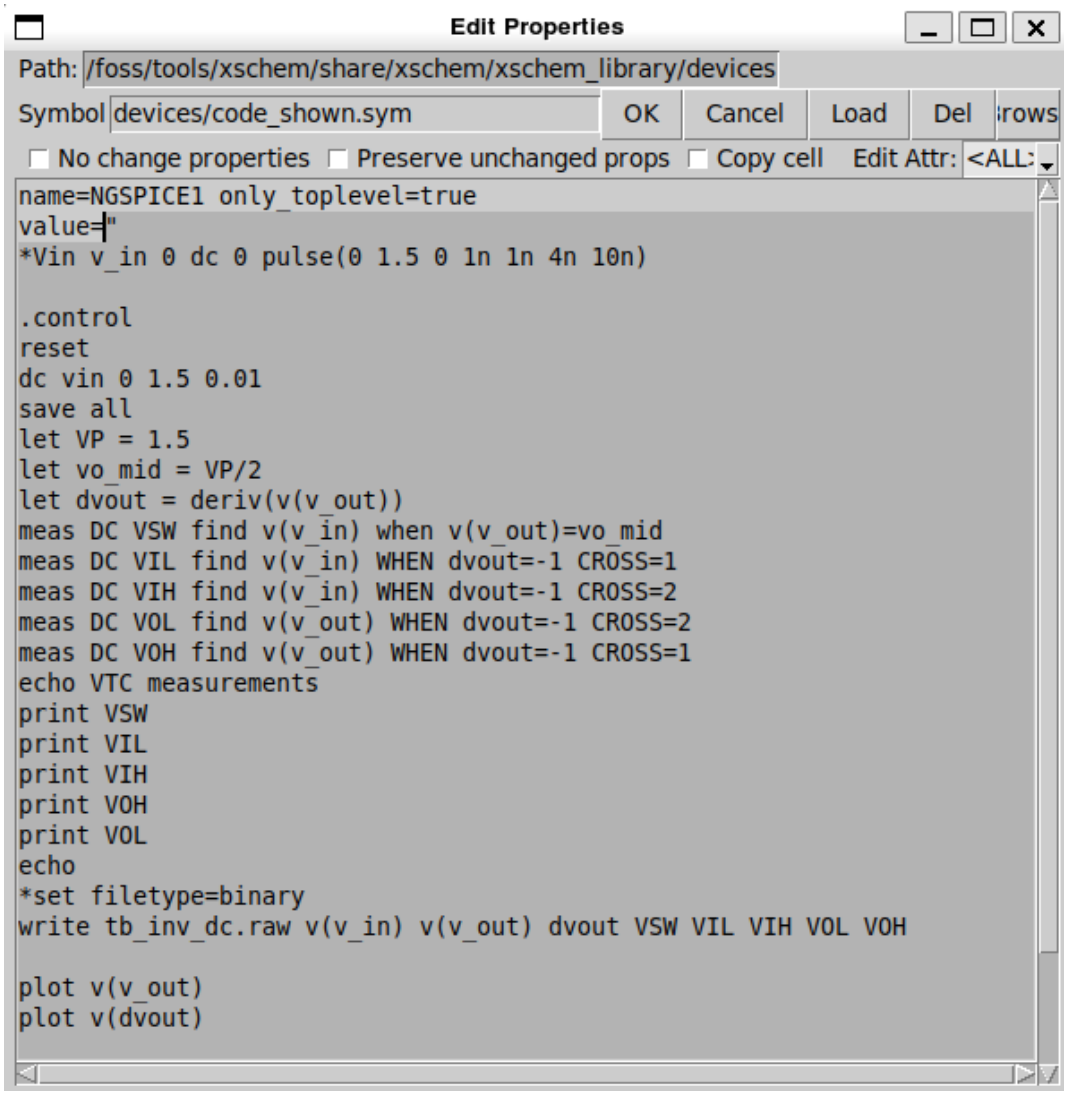


Figure 5.6.: Xschem Testbench.

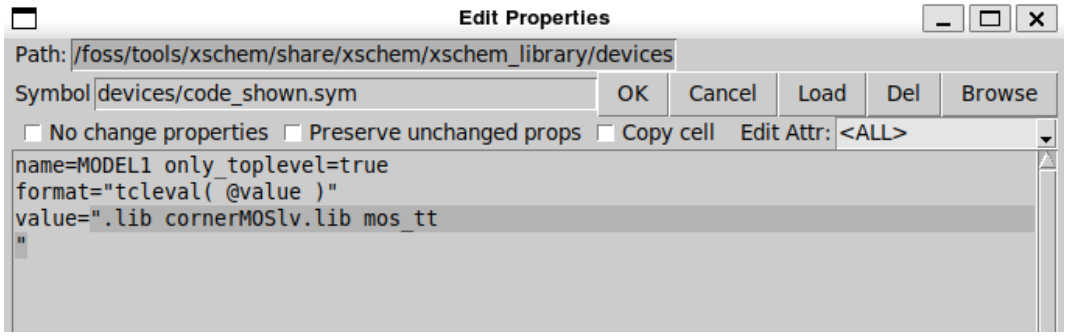


Figure 5.7.: Xschem TT Corner.

Important

We have used NgSpice interactive mode for the spice simulation see Figure 5.8. We can also find other modes `Simulation --> Configure simulator and`

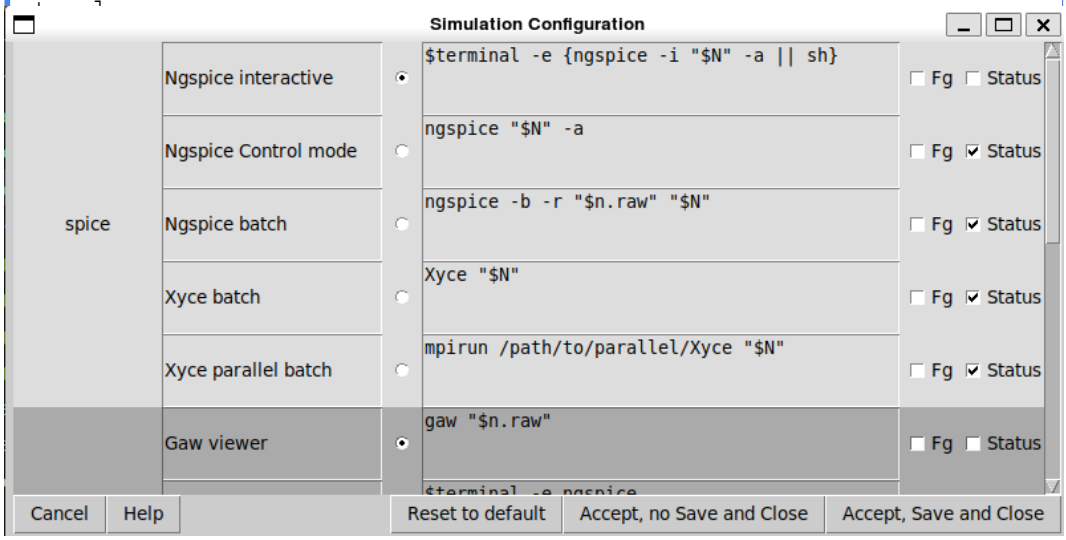


Figure 5.8.: Simulator Mode

5.3. Output Results

Outputs of dc and transient analysis can be seen in

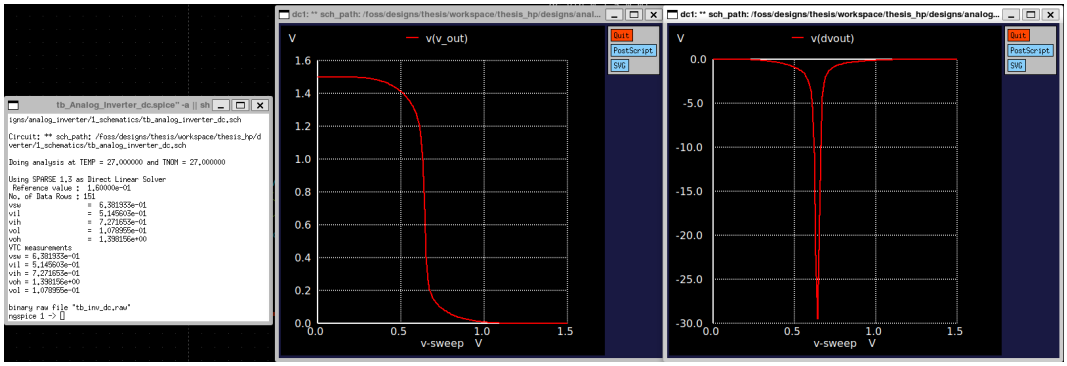


Figure 5.9.: DC Analysis

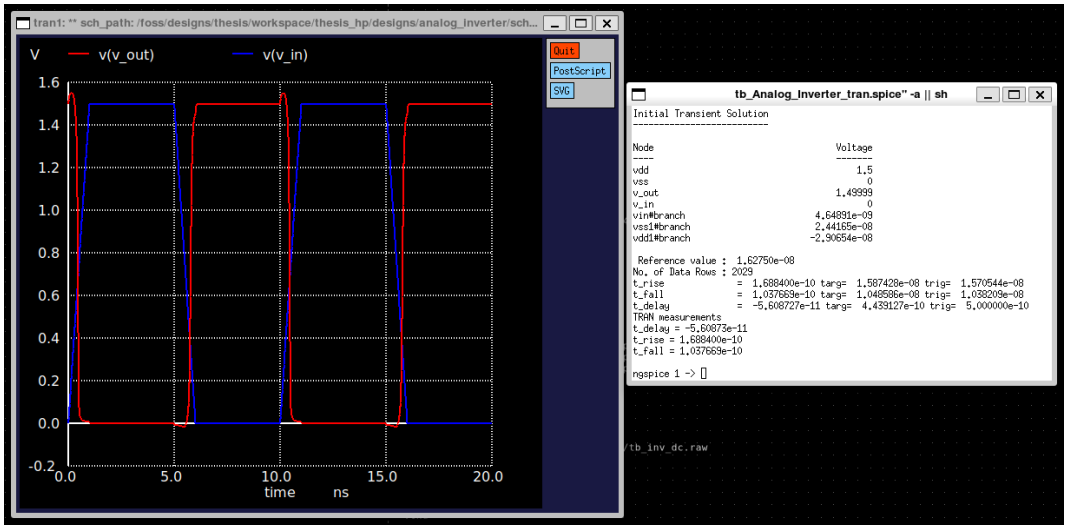


Figure 5.10.: Transient Analysis

5.4. Post processing and simulation using python

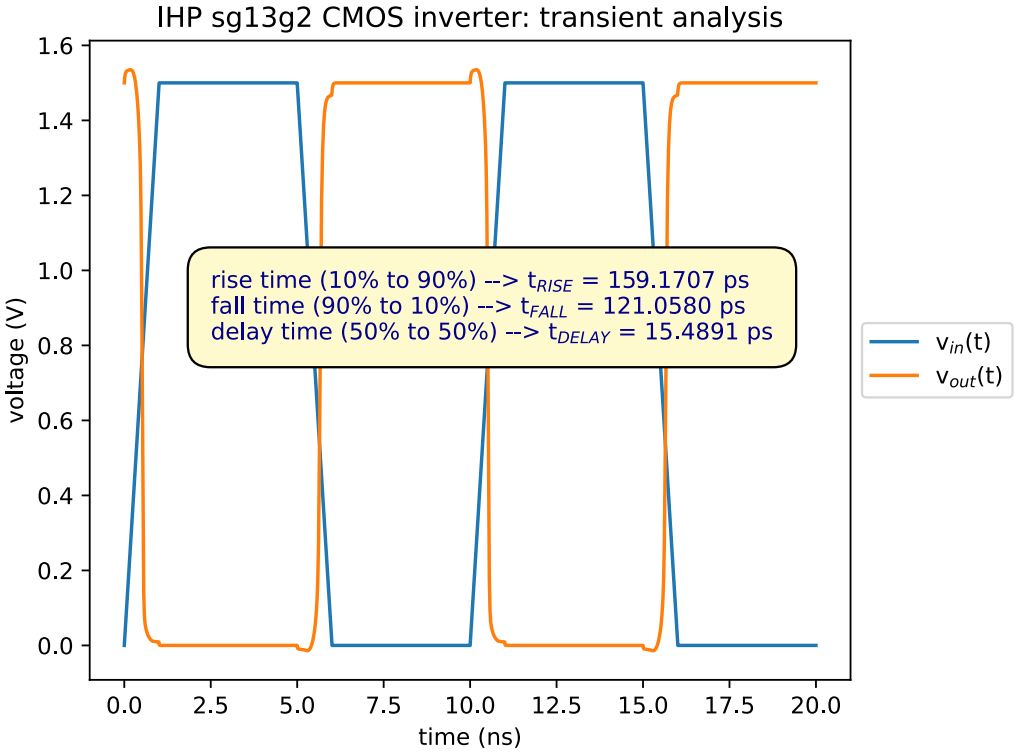


Figure 5.11.: Transient Curver

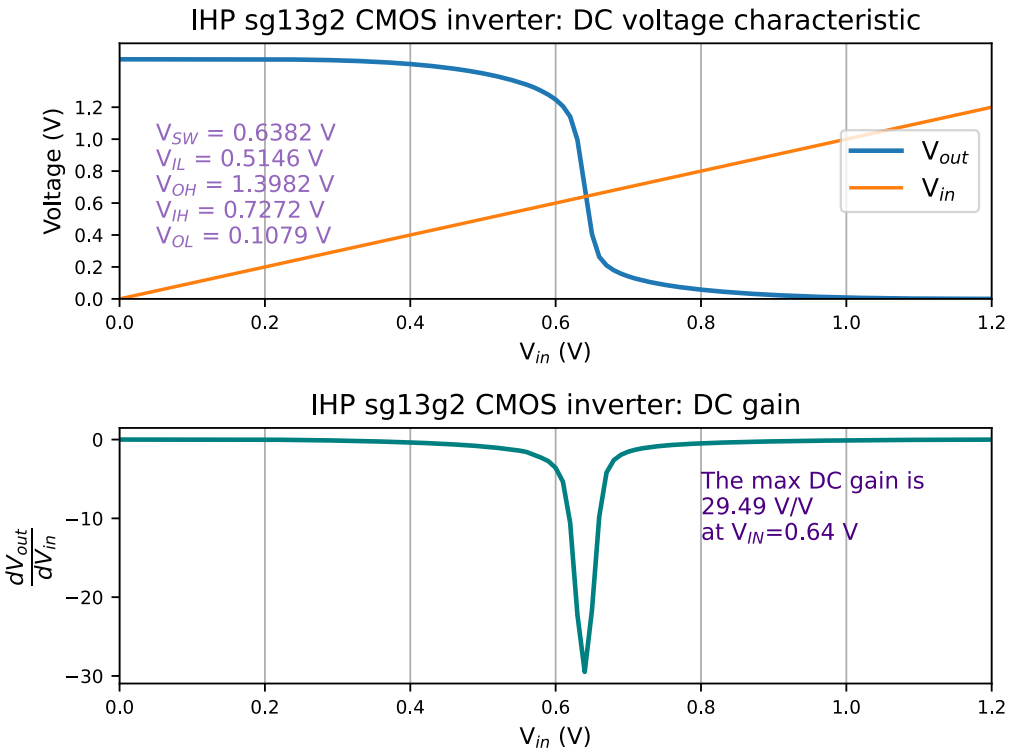


Figure 5.12.: DC Charachteristics

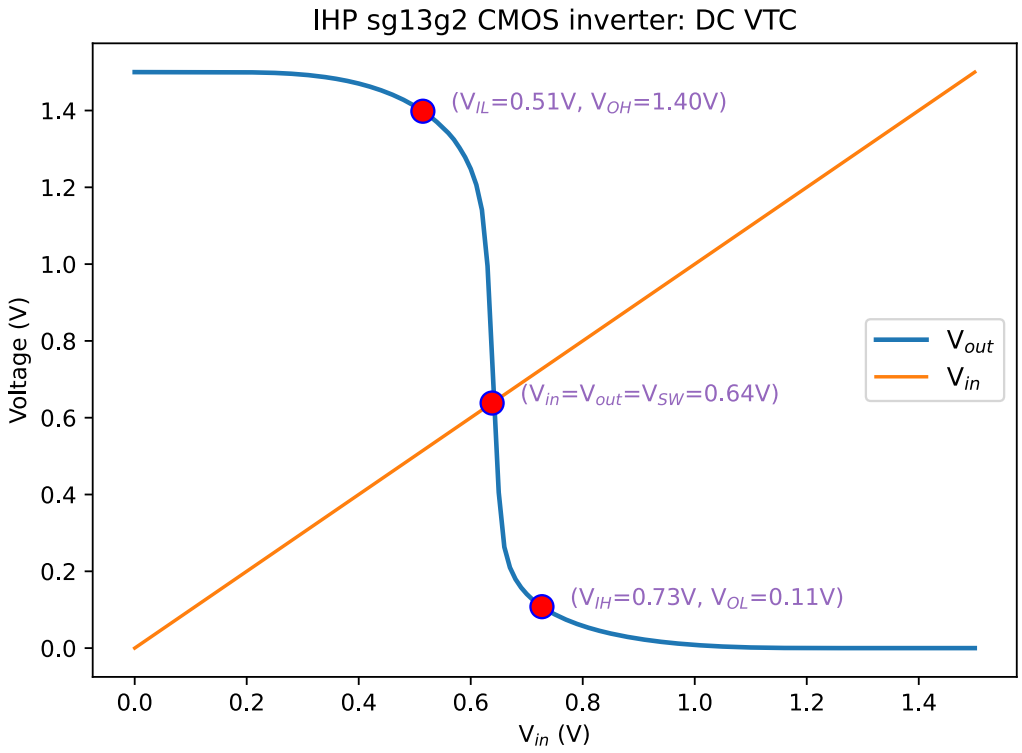


Figure 5.13.: VTC

5.5. Introduction to Layout

Layout is the physical representation of an integrated circuit, where circuit components such as transistors, interconnects, and vias are defined geometrically on various layers corresponding to materials used in fabrication. The layout must adhere strictly to design rules provided by the semiconductor foundry. We have two tools **Magic** and **KLayout** which enables layout design and verification. Magic being a long-standing tool favored for its tight integration with open-source flows, while KLayout offers powerful editing, scripting, and visualization capabilities, especially suited for hierarchical designs and PDKs such as SG13G2.

5.5.1. Designing Layout using Klayout

Layout design is performed using KLayout, an open-source layout viewer and editor for GDSII and OASIS files. Inverter layout can be built either **manually from scratch or using parametric cells (P_Cells)** from the IHP SG13G2 PDK, p_cells simplify device instantiation by allowing parameterized generation of transistors and other structures. KLayout supports precise layer control, design rule checking, and layout-versus-schematic (LVS) compatibility.

1. Start terminal and run IIC-OSIC docker image.
2. To select the PDK, use command `iic-pdk ihp-sg13g` refer Figure 5.14

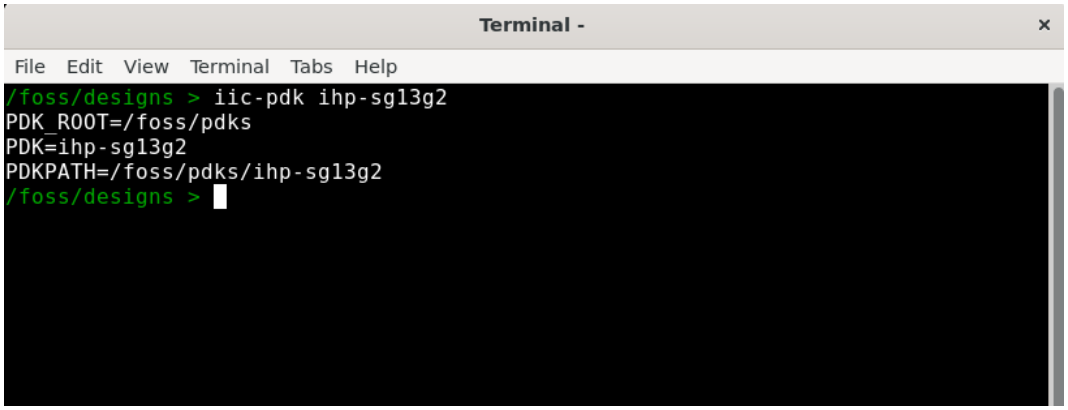


Figure 5.14.: Technology Selection

> **Important**
>
> This step is crucial as it sets the technology IHP-SG13G2 in the tools.

3. Start klayout amd refer Figure 5.15. A new window of Klayout will open after the command.

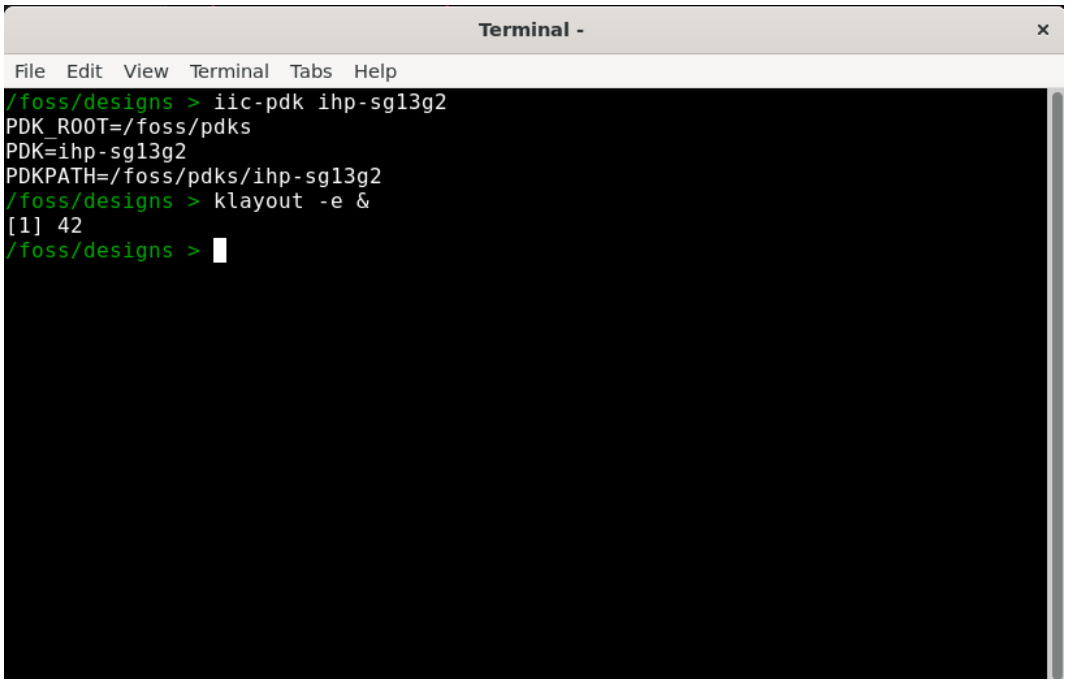


Figure 5.15.: Klayout Launch.

4. To create new layout, Go to `Files >> New Layout`. New Layout Properties dialog box will appear, change `Top Cell from TOP to inv(inverter)` and leave the rest the same refer Figure 5.16

5. Now go to `Basic - Basic Layout Objects` on the left panel. Under `SG13_dev - IHP SG13G2 Pcells`, select an NMOS device and place it in

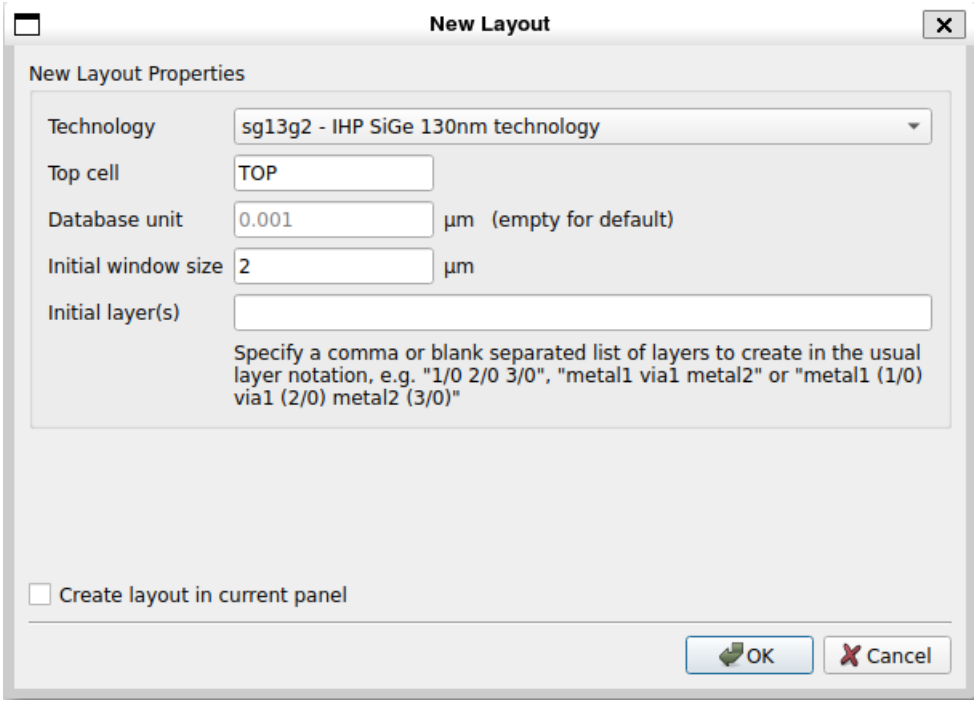


Figure 5.16.: New Layout

the layout. `Press ESC` to exit placement mode. Repeat the process for the PMOS, placing it directly above the NMOS to follow standard inverter structure.

6. On the right side, the layer list shows all available layers; bold entries indicate layers currently used in the layout. To view all layers from the full cell hierarchy, `click Display → Full Hierarchy | Press Shift + F`.

> **Tips**
>
> To rotate a PCell, go to the `Edit tab >> Move`, select the PCell, then right-click. Use the mouse wheel to zoom in and out, and press the middle mouse button to pan the layout view.

7. `Double-click on the PCell` to open its parameter editor. Set parameters like the width and length values to match those used in your **Xschem schematic**, then `click OK` to apply the changes refer Figure 5.18

8. Use the keyboard arrow keys to align the NMOS directly below the PMOS. Connect the **input** by joining the gates of both transistors using the `polysilicon layer – GatPoly.drawing`. Connect the **output** by linking the source and drain of PMOS and NMOS with the `Metal1 layer – Metal1.drawing`. Similarly, use `Metal1.drawing` to connect the PMOS source to VCC and the NMOS source to VSS. Label all metal connections using the `Metal1 pin layer – Metal1.pin`. Refer to the IHP Open Source PDK DesignLIB documentation to verify the correct layer numbers and purposes.

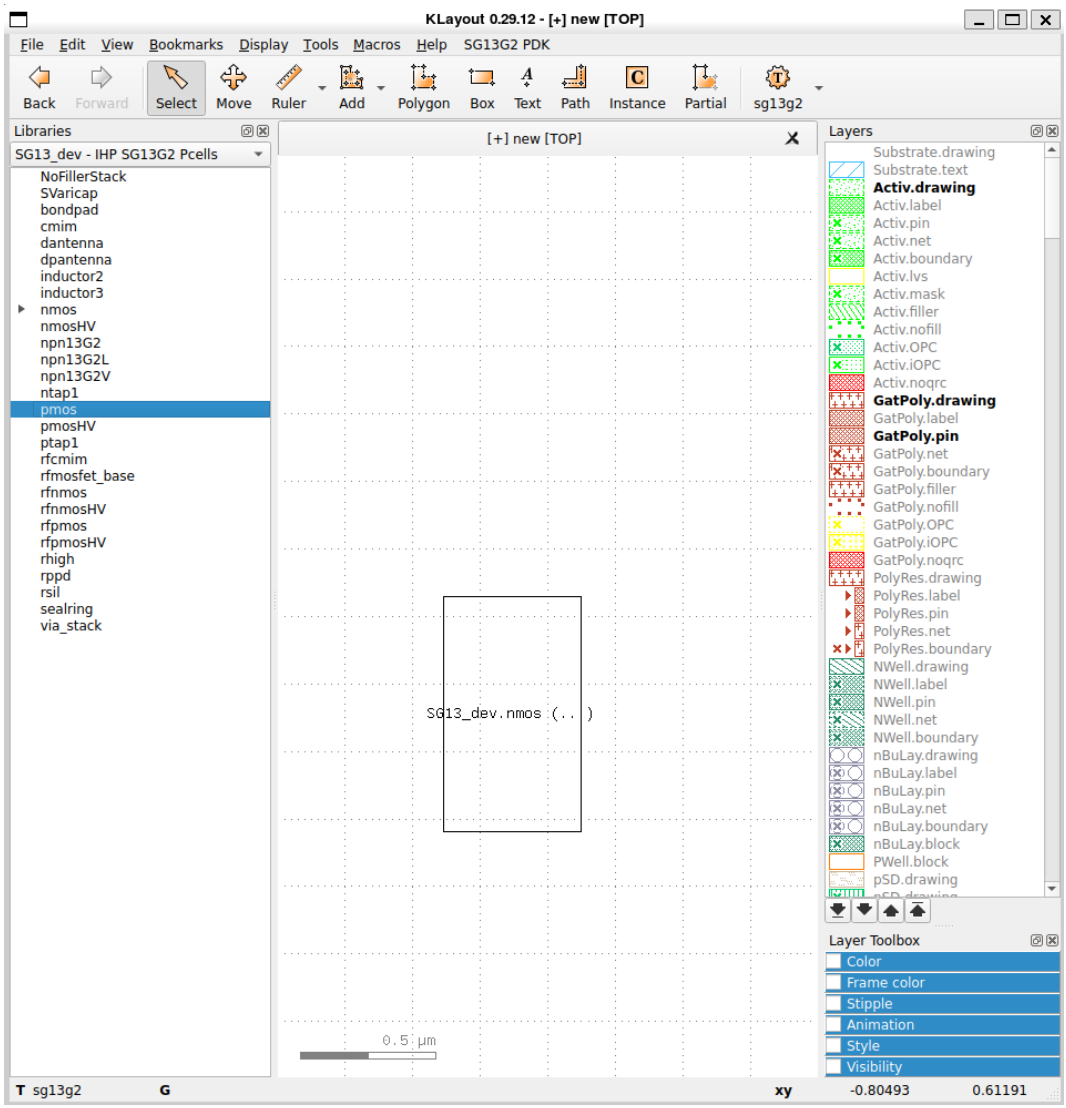


Figure 5.17.: Pcells

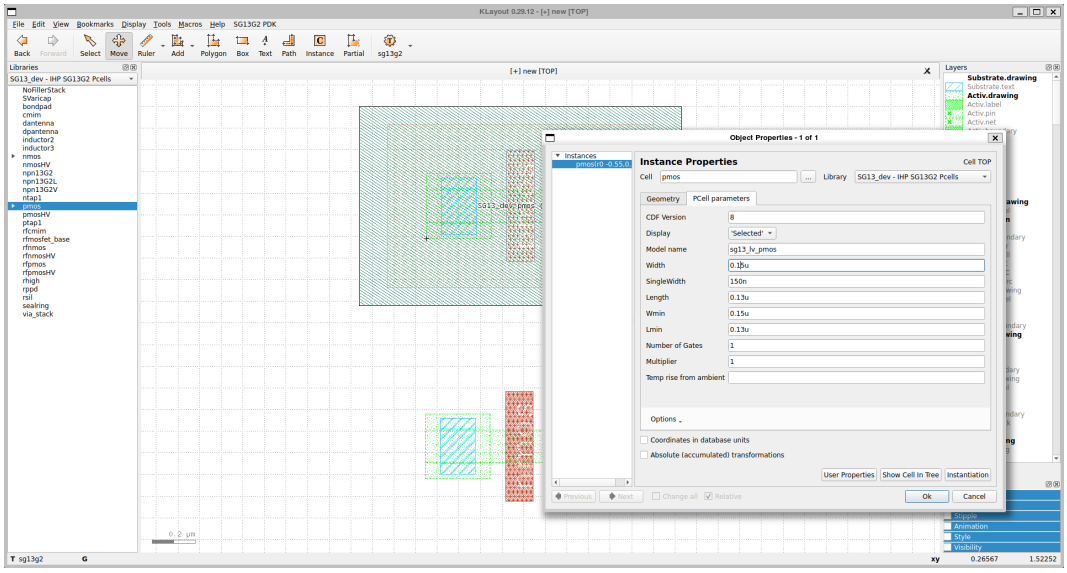


Figure 5.18.: Parameters Window

9. Finally, save your layout by going to `File → Save As`, and choose the `GDSII` format to export your design. Your completed CMOS inverter layout should resemble the image shown below.

Here you can find all the layout rules in depth and with illustrations.

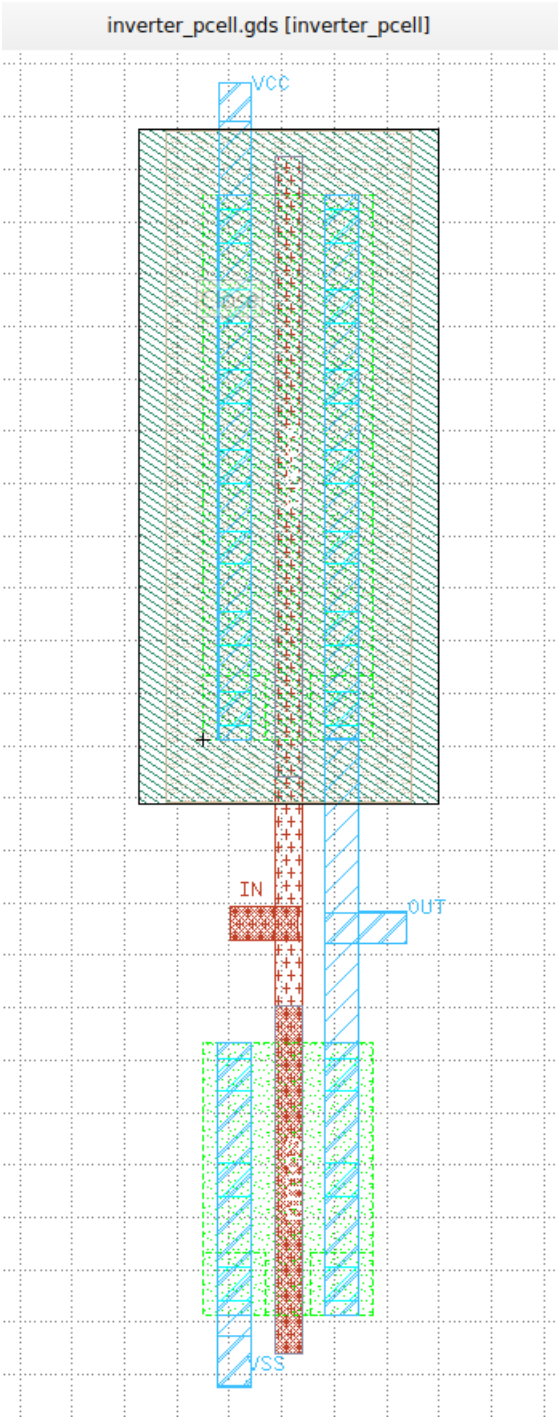


Figure 5.19.: Final Layout

6. Klayout-PEX (KPEX)

While in the schematic, a connection between device terminals is seen as an equipotential, the stacked geometries in a specific layout introduce parasitic effects, which can be thought of additional resistors, capacitors (and inductors), not modeled by and missing in the original schematic.

To be able to simulate these effects, a parasitic extraction tool (PEX) is used, to extract a netlist from the layout, which represents the original schematic (created from the layout active and passive elements) augmented with the additional parasitic devices.

6.1. Why Do We Need PEX?

Reason	Description
Performance Degradation	Parasitic capacitance reduces bandwidth and phase margin.
Offset and Mismatch Effects	Parasitic coupling causes imbalance in differential paths.
Stability Impact	Additional phase lag from parasitics can destabilize feedback loops.
Post-Layout Verification	Confirms that layout did not violate design intent (gain, UGB, PSRR, etc.).
Tapeout Confidence	Ensures high correlation between simulation and silicon behavior.

IIC-OSIC-Tools offers KPEX for this purpose which is a tool, well integrated with Klayout by using its API.

6.2. PEX using Klayout-PEX tool

IIC-OSIC-Tools comes with pre-installed tool called as **K-pex**. The current status of KLayout-PEX says as following:

Available KLayout PEX Engines:

Engine	PEX Type	Status	Description
`KPEX/MAGIC`	CC, RC	Usable	Wrapper engine, using installed `magic` tool

Engine	PEX Type	Status	Description
KPEX/FasterCap	CC	Usable, *pending QA*	Field solver engine using FasterCap
KPEX/FastHenry2	RL	Planned	Field solver engine using FastHenry2
KPEX/2.5D	CC	Under construction	Prototype engine implementing MAGIC concepts/formulas with KLayout means
KPEX/2.5D	R, RC	Planned	Prototype engine implementing MAGIC concepts/formulas with KLayout means

6.3. Running the KPEX/MAGIC Engine

The magic section of `kpex --help` describes the arguments and their defaults. Important arguments:

- `--magicrc`: specify location of the `magicrc` file
- `--gds`: path to the GDS input layout
- `--magic`: enable magic engine
- `--out_dir` : set the output directory

6.3.1. Example Command

```
kpex --pdk ihp_sg13g2 --magic --gds GDS_PATH --out_dir OUTPUT_DIR_P
```

more to kpex can be found under the command `kpex --help`

```
/foss/designs > kpex --help
Usage: kpex [--help] [--version] [--log_level LOG_LEVEL]
            [--threads NUM_THREADS] --pdk {ihp_sg13g2,sky130A}
            [--out_dir OUTPUT_DIR_BASE_PATH] [--gds GDS_PATH]
            [--schematic SCHEMATIC_PATH] [--lvsdb LVSDB_PATH]
            [--cell CELL_NAME] [--cache-lvs CACHE_LVS]
            [--cache-dir CACHE_DIR_PATH] [--lvs-verbose KLAYOUT_LVS
            [--blackbox BLACKBOX_DEVICES] [--fastercap] [--fastcap]
            [--2.5D] [--k_void K_VOID] [--delaunay_amax DELAUNAY_AM
            [--delaunay_b DELAUNAY_B] [--geo_check GEOMETRY_CHECK]
            [--diel DIELECTRIC_FILTER] [--tolerance FASTERCAP_TOLER
            [--d_coeff FASTERCAP_D_COEFF]
            [--mesh FASTERCAP_MESH_REFINEMENT_VALUE]
            [--ooc FASTERCAP_OOC_CONDITION]
            [--auto_precond FASTERCAP_AUTO_PRECONDITIONER] [--galer
            [--jacobi] [--magicrc MAGICRC_PATH] [--magic_mode {CC,R
            [--magic_cthresh MAGIC_CTHRESH] [--magic_rthresh MAGIC_
```

```
            [--magic_tolerance MAGIC_TOLERANCE] [--magic_halo MAGIC_HAL
            [--magic_short {none,resistor,voltage}]
            [--magic_merge {none,conservative,aggressive}] [--mode {CC,
            [--halo HALO] [--scale SCALE_RATIO_TO_FIT_HALO]

kpex: KLayout-integrated Parasitic Extraction Tool

Special Options:
  --help, -h            show this help message and exit
  --version, -v         show program's version number and exit
  --log_level LOG_LEVEL
                        log_level □ {'all', 'debug', 'subprocess', 'ver
                        'info', 'warning', 'error', 'critical'}. Defaul
                        'subprocess'
  --threads NUM_THREADS
                        number of threads (e.g. for FasterCap) (default

Parasitic Extraction Setup:
  --pdk {ihp_sg13g2,sky130A}
                        pdk □ {'ihp_sg13g2', 'sky130A'}
  --out_dir, -o OUTPUT_DIR_BASE_PATH
                        Output directory path (default is 'output')

Parasitic Extraction Input:
  Either LVS is run, or an existing LVSDB is used

  --gds, -g GDS_PATH    GDS path (for LVS)
  --schematic, -s SCHEMATIC_PATH
                        Schematic SPICE netlist path (for LVS). If none
                        a dummy schematic will be created
  --lvsdb, -l LVSDB_PATH
                        KLayout LVSDB path (bypass LVS)
  --cell, -c CELL_NAME  Cell (default is the top cell)
  --cache-lvs CACHE_LVS
                        Used cached LVSDB (for given input GDS) (defaul
                        True)
  --cache-dir CACHE_DIR_PATH
                        Path for cached LVSDB (default is .kpex_cache w
                        --out_dir)
  --lvs-verbose KLAYOUT_LVS_VERBOSE
                        Verbose KLayout LVS output (default is False)

Parasitic Extraction Options:
  --blackbox BLACKBOX_DEVICES
                        Blackbox devices like MIM/MOM caps, as they are
                        handled by SPICE models (default is False for t
                        now)
  --fastercap           Run FasterCap engine (default is False)
```

```
  --fastcap             Run FastCap2 engine (default is False)
  --magic               Run MAGIC engine (default is False)
  --2.5D                Run 2.5D analytical engine (default is Fals

Fastercap Options:
  --k_void, -k K_VOID   Dielectric constant of void (default is 3.9
  --delaunay_amax, -a DELAUNAY_AMAX
                        Delaunay triangulation maximum area (defaul
  --delaunay_b, -b DELAUNAY_B
                        Delaunay triangulation b (default is 0.5)
  --geo_check GEOMETRY_CHECK
                        Validate geometries before passing to Faste
                        (default is False)
  --diel DIELECTRIC_FILTER
                        Comma separated list of dielectric filter p
                        Allowed patterns are: (none, all, -dielname
                        +dielname2) (default is all)
  --tolerance FASTERCAP_TOLERANCE
                        FasterCap -aX error tolerance (default is 0
  --d_coeff FASTERCAP_D_COEFF
                        FasterCap -d direct potential interaction c
                        to mesh refinement (default is 0.5)
  --mesh FASTERCAP_MESH_REFINEMENT_VALUE
                        FasterCap -m Mesh relative refinement value
                        is 0.5)
  --ooc FASTERCAP_OOC_CONDITION
                        FasterCap -f out-of-core free memory to lin
                        condition (0 = don't go OOC, default is 2)
  --auto_precond FASTERCAP_AUTO_PRECONDITIONER
                        FasterCap -ap Automatic preconditioner usag
                        is True)
  --galerkin            FasterCap -g Use Galerkin scheme (default i
  --jacobi              FasterCap -pj Use Jacobi preconditioner (de
                        False)

Magic Options:
  --magicrc MAGICRC_PATH
                        Path to magicrc configuration file (default
                        '/foss/pdks/ihp-sg13g2/libs.tech/magic/ihp-
                        crc')
  --magic_mode {CC,RC,R}
                        magic_mode □ {'CC', 'RC', 'R'}. Defaults to
  --magic_cthresh MAGIC_CTHRESH
                        Threshold (in fF) for ignored parasitic capa
                        (default is 0.01). (MAGIC command: ext2spic
                        <value>)
  --magic_rthresh MAGIC_RTHRESH
                        Threshold (in Ω) for ignored parasitic resis
```

```
                        (default is 100). (MAGIC command: ext2spice rth
                        <value>)
  --magic_tolerance MAGIC_TOLERANCE
                        Set ratio between resistor and device tolerance
                        (default is 1). (MAGIC command: extresist toler
                        <value>)
  --magic_halo MAGIC_HALO
                        Custom sidewall halo distance (in µm) (MAGIC co
                        extract halo <value>) (default is no custom hal
  --magic_short {none,resistor,voltage}
                        magic_short □ {'none', 'resistor', 'voltage'}.
                        Defaults to 'none'
  --magic_merge {none,conservative,aggressive}
                        magic_merge □ {'none', 'conservative', 'aggress
                        Defaults to 'none'

2.5D Options:
  --mode {CC,RC,R}      mode □ {'CC', 'RC', 'R'}. Defaults to 'CC'
  --halo HALO           Custom sidewall halo distance (in µm) to overri
                        info (default is no custom halo)
  --scale SCALE_RATIO_TO_FIT_HALO
                        Scale fringe ratios, so that halo distance is 1
                        (default is True)

Environmental variables:

  Variable              Description
□□□
```

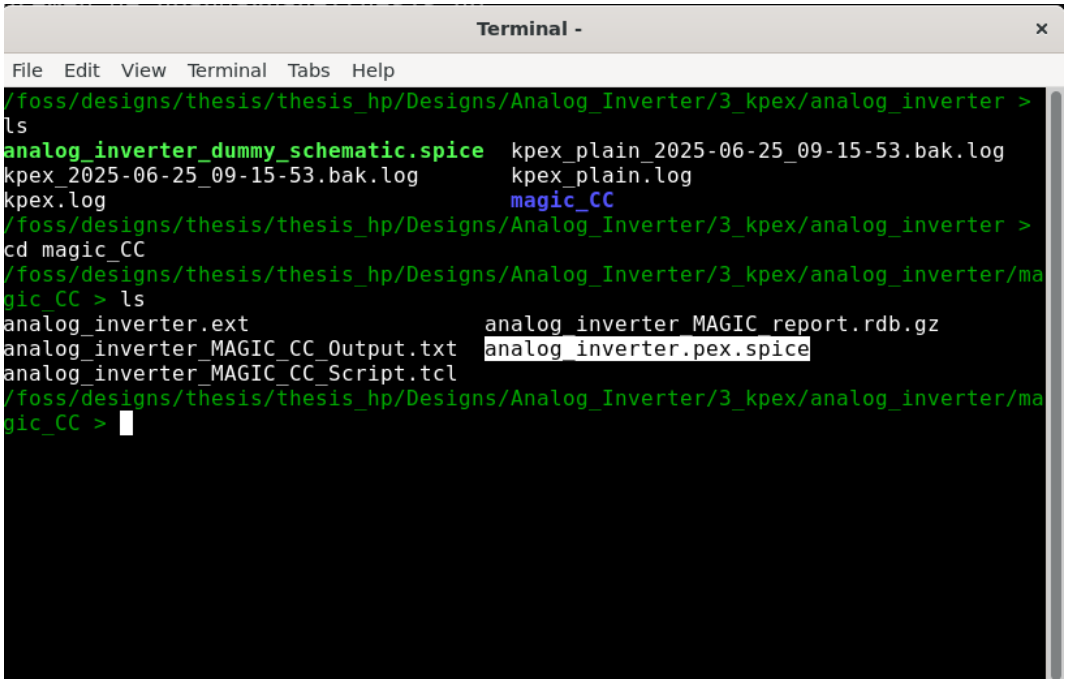


Figure 6.1.: Extracted Spice file

6.4. Post layout simulation.

We have established testbenches for DC, AC, and transient analysis using the symbol of the designed inverter as a subcircuit. The next step is to perform post-layout simulation by replacing the schematic netlist with the extracted layout netlist (PEX), in order to verify whether the performance remains consistent after layout parasitics are included.

Firstly we need to open the symbol file in Xschem, edit the properties of the symbol See Figure 6.2. Change from `subcircuit` to `primitive`.

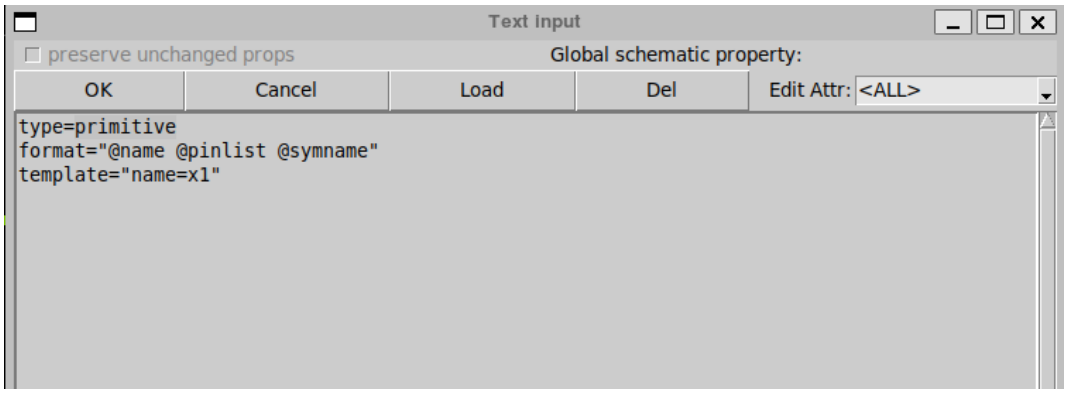


Figure 6.2.: Primitive

Once changed to primitive, now we can include the `.spice` file from PEX to the testbench. See Figure 6.3

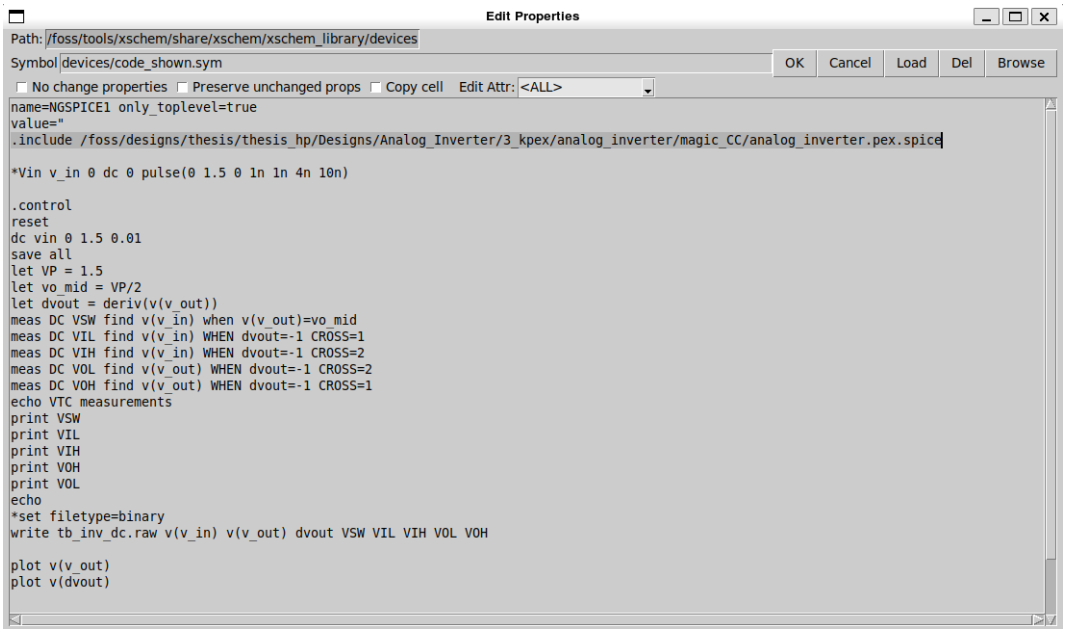


Figure 6.3.: Include

6.5. Results

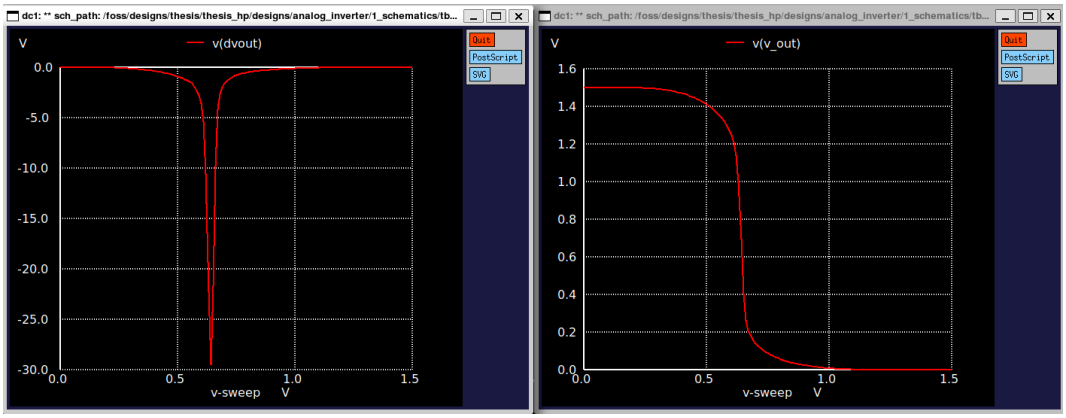


Figure 6.4.: Result

Figure 6.4 shows that the response is as same as we expected from our schematic.

7. Xschem Commands

Useful shortcuts are as follows

Moving around in a schematic:

- `Cursor keys` to move around
- `Ctrl-e` to go back to parent schematic
- `e` to descend into schematic of selected symbol
- `i` to descend into symbol of selected symbol
- `f` full zoom on schematic
- `Shift-z` to zoom in
- `Ctrl-z` to zoom out

Editing schematics:

- `Del` to delete elements
- `Ins` to insert elements from library
- `Escape` to abort an operation
- `Ctrl-#` to rename components with duplicate names
- `c` to copy elements
- `Alt-Shift-l` to add wire label
- `Alt-l` to add label pin
- `m` to move selected objects
- `Shift-R` to rotate selected objects
- `Shift-F` to mirror / flip selected objects
- `q` to edit properties
- `Ctrl-s` to save schematic
- `t` to place a text
- `Shift-T` to toggle the `ignore` flag on an instance
- `u` to undo an operation
- `w` to draw a wire
- `Shift-W` draw wire and snap to close pin or net point
- `&` to join, break, and collapse wires
- `A` to make symbol from schematic
- `Alt-s` to reload the circuit if changes in a subcircuit were made

Viewing/Simulating Schematics

- `5` to only view probes
- `k` to highlight selected net
- `Shift-K` to unhighlight all nets
- `Shift-o` to toggle light/dark color scheme
- `s` to run a simulation
- `a & b` to add cursors to an in-circuit simulation graph
- `f` full zoom on y- or x-axis in in-circuit simulation graph

8. ngspice Commands

Useful shortcuts are as follows:

8.1. Commands

- `ac dec|lin points fstart fstop` performs a small-signal ac analysis with either linear or decade sweep
- `dc sourcename vstart vstop vincr [src2 start2 stop2 incr2]` runs a dc-sweep, optionally across two variables
- `display` shows the available data vectors in the current plot
- `echo` can be used to display text, `$variable` or `$&vector`, can be useful for debugging
- `let name = expr` to create a new vector; `unlet vector` deletes a specified vector; access vector data with `$&vec`
- `linearize vec` linearizes a vector on an equidistant time scale, do this before an FFT; with `set specwindow=windowtype` a proper windowing function can be set
- `meas` can be used for various evaluations of measurement results (see ngspice manual for details)
- `noise v(output <ref>) src (dec|lin) pts fstart fstop` runs a small-signal noise analysis
- `op` calculates the operating point, useful for checking bias points and device parameters
- `plot expr vs scale` to plot something
- `print expr` to print it, use `print all` to print everything
- `remzerovec` can be useful to remove vectors with zero length, which otherwise cause issues when saving or plotting data
- `rusage` plot information about resource usage like memory
- `save all` or `save signal` specifies which data is saved during simulation; this lowers RAM usage during simulation and size of RAW file; do save before the actual simulation statement
- `setplot` show a list of available plots
- `set var = value` to set the value of a variable; use variable with `$var`; `unset var` removes a variable
- `set enable_noisy_r` to enable noise of behavioral resistors; usually, this is a good idea
- `shell cmd` to run a shell command
- `show : param`, like `show : gm` shows the g_m of all devices after running an operating point with `op`

- `spec` plots a spectrum (i.e. frequency domain plot)
- `status` shows the saved parameters and nodes
- `tf` runs a transfer function analysis, returning transfer function, input and output resistance
- `tran tstep tstop <tstart <tmax>>` runs a transient analysis until `tstop`, reporting results with `tstep` step size, starting to plot at `tstart` and performs time steps not larger then `tmax`
- `wrdata` writes data into a file in a tabular ASCII format; easy to further process
- `write` writes simulation data (the saved nodes) into a RAW file; default is binary, can be changed to ASCII with `set filetype=ascii`; with `set appendwrite` data is added to an existing file

8.2. Options

Use `option option=val option=val` to set various options; important ones are:

- `abstol` sets the absolute current error tolerance (default is 1pA)
- `gmin` is the conductance applied at every node for convergence improvement (default is 1e-12); this can be critical for very high impedance circuits
- `klu` sets the KLU matrix solver
- `list` print the summary listing of the input data
- `maxord` sets the numerical order of the integration method (default is 2 for Gear)
- `method` set the numerical integration method to `gear` or `trap` (default is `trap`)
- `node` prints the node table
- `opts` prints the option values
- `temp` sets the simulation temperature
- `reltol` set the relative error tolerance (default is 0.001 = 0.1%)
- `savecurrents` saves the terminal currents of all devices
- `sparse` sets the sparse matrix solver, which can run noise analysis, but is slower than `klu`
- `vntol` sets the absolute voltage error tolerance (default is 1µV)
- `warn` enables the printing of the SOA warning messages

8.3. Convergence Helper

- `option gmin` can be used to increase the conductance applied at every node
- `option method=gear` can lead to improved convergence
- `.nodeset` can be used to specify initial node voltage guesses
- `.ic` can be used to set initial conditions

Bibliography

[1] I. Arnaldi, *Design of sigma-delta converters in MATLAB®/simulink®*. Springer International Publishing, 2019. doi: 10.1007/978-3-319-91539-5.

[2] O. Bajdechi and J. Huijsing, *Systematic design of sigma-delta analog-to-digital converters*, vol. 768. in The springer international series in engineering and computer science, vol. 768. Kluwer Academic Publishers, 2004, p. 208. Available: http://www.springer.com/engineering/circuits+%26+systems/book/978-1-4020-7945-0?changeHeader

[3] R. J. Baker, "CMOSedu.com." Accessed: Nov. 13, 2025. [Online]. Available: https://cmosedu.com

[4] R. J. Baker, *CMOS: Mixed-signal circuit design*, 2nd ed. Wiley-IEEE Press, 2008, p. 352. doi: 10.13140/RG.2.1.5146.6005.

[5] R. J. Baker, *CMOS: Circuit design, layout, and simulation*, 3rd ed. Wiley, 2010. doi: 10.1002/9780470891179.

[6] B. Baker, "How delta-sigma ADCs work, Part 1," Texas Instruments Incorporation, Analog Application Journal (AAJ) SLYT423a, 2011. Available: https://www.ti.com/lit/an/slyt423a/slyt423a.pdf

[7] B. Baker, "How delta-sigma ADCs work, Part 2," Texas Instruments Incorporation, Analog Application Journal (AAJ) SLYT438, 2011. Available: https://www.ti.com/lit/an/slyt438/slyt438.pdf

[8] N. Blachman, "The intermodulation and distortion due to quantization of sinusoids," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 33, no. 6, pp. 1417–1426, 1985, doi: 10.1109/TASSP.1985.1164729.

[9] B. E. Boser and B. A. Wooley, "The design of sigma-delta modulation analog-to-digital converters," *IEEE Journal of Solid-State Circuits*, vol. 23, no. 6, pp. 1298–1308, Dec. 1988, doi: 10.1109/4.90025.

[10] B. E. Boser, "EECS247: Analog-digital interface integrated circuits." Course notes, University of California Berkeley, Feb. 2003.

[11] B. P. Brandt and B. A. Wooley, "A low-power, area-efficient digital filter for decimation and interpolation," *IEEE Journal of Solid-State Circuits*, vol. 29, no. 6, pp. 679–687, June 1994, doi: 10.1109/4.293113.

[12] J. Candy, "Decimation for sigma delta modulation," *IEEE Transactions on Communications*, vol. 34, no. 1, pp. 72–76, Jan. 1986, doi: 10.1109/TCOM.1986.1096432.

[13] Y. Chae *et al.*, "A 2.1 m pixels, 120 frame/s CMOS image sensor with column-parallel $\Delta\Sigma$ ADC architecture," *IEEE Journal of Solid-State Circuits*, vol. 46, no. 1, pp. 236–247, 2011, doi: 10.1109/JSSC.2010.2085910.

[14] M. Clifford, "Fundamental principles behind the sigma-delta ADC topology: Part 2," Analog Devices (AD), Feb. 2016. Available: https://www.analog.com/en/resources/technical-articles/fundamental-principles-behind-sigma-delta-adc-topology-part2.html

[15] M. Clifford, “Fundamental principles behind the sigma-delta ADC topology: Part 1,” Analog Devices (AD), Jan. 2016. Available: https://www.analog.com/en/resources/technical-articles/behind-the-sigma-delta-adc-topology.html

[16] P. R. Gray and R. G. Meyer, “MOS operational amplifier design-a tutorial overview,” *IEEE Journal of Solid-State Circuits*, vol. 17, no. 6, pp. 969–982, Dec. 1982, doi: 10.1109/JSSC.1982.1051851.

[17] K. Herman *et al.*, “On the versatility of the IHP BiCMOS open source and manufacturable PDK: A step towards the future where anybody can design and build a chip,” *IEEE Solid-State Circuits Magazine*, vol. 16, no. 2, pp. 30–38, 2024, doi: 10.1109/mssc.2024.3372907.

[18] R. C. Jaeger and T. N. Blalock, *Microelectronic circuit design*, 6th ed. New York, New York: McGraw-Hill, 2023. Available: http://ebookcentral.proquest.com/lib/suub/detail.action?docID=6865812

[19] W. Kester, *Data conversion handbook*. Newnes, 2004. Available: http://www.analog.com/library/analogDialogue/archives/39-06/data_conversion_handbook.html

[20] J. Lienig and J. Scheible, *Fundamentals of layout design for electronic circuits*. Springer International Publishing, 2020. doi: 10.1007/978-3-030-39284-0.

[21] J. Lienig and J. Scheible, *Grundlagen des layoutentwurfs elektronischer schaltungen*. Springer International Publishing, 2023. doi: 10.1007/978-3-031-15768-4.

[22] R. Lyons, “Optimizing the half-band filters in multistage decimation and interpolation.” DSP Related, Jan. 2016. Available: https://www.dsprelated.com/showarticle/903.php

[23] R. Lyons, “A beginner’s guide to cascaded integrator-comb (CIC) filters.” DSP Related Blog, Mar. 2020. Available: https://www.dsprelated.com/showarticle/1337.php

[24] J. Markus, J. Silva, and G. C. Temes, “Theory and applications of incremental $\Delta\Sigma$ converters,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 51, no. 4, pp. 678–690, 2004, doi: 10.1109/TCSI.2004.826202.

[25] Maxim, “Demystifying delta-sigma ADCs,” Maxim Integrated TM, Tutorial 1870, Jan. 2003. Available: https://www.maximintegrated.com/en/app-notes/index.mvp/id/1870

[26] B. Murmann, *Analysis and design of elementary MOS amplifier stages*. in The modular series of microelectronic device and circuit design. NTS Press, 2013. Available: https://github.com/bmurmann/Book-on-MOS-stages/blob/main/book/Analysis and Design of Elementary MOS Amplifier Stages.pdf

[27] B. Murmann, “EE 628: Analysis and design of integrated circuits.” GitHub, University of Hawai‘i at Mānoa, 2024. Available: https://github.com/bmurmann/EE628

[28] S. Pavan, R. Schreier, and G. C. Temes, *Understanding delta-sigma data converters*. in IEEE press series on microelectronic systems. Wiley Press, 2017, p. 584. doi: 10.1002/9781119258308.

[29] R. van de Plassche and R. E. J. van der Grift, “A five-digit analog-digital converter,” *IEEE Journal of Solid-State Circuits*, vol. 12, no. 6, pp. 656–662, 1977, doi: 10.1109/JSSC.1977.1050975.

[30] H. Pretl and M. Eberlein, “Fifty nifty variations of two-transistor circuits: A tribute to the versatility of MOSFETs,” *IEEE Solid-State Circuits Magazine*, vol. 13, no. 3, pp. 38–46, 2021, doi: 10.1109/mssc.2021.3088968.

[31] H. Pretl, M. Koefinger, and S. Dorrer, “Analog Circuit Design.” Dec. 2024. doi: 10.5281/zenodo.14387481.

[32] H. Pretl and G. Zachl, *GitHub repository of the IIC-OSIC-TOOLS*. (Jan. 2025). Zenodo. doi: 10.5281/zenodo.14634518.

[33] P. Rambo, V. Sharma, and C. Wittke, “IHP analog academy.” Accessed: Nov. 13, 2025. [Online]. Available: https://github.com/IHP-GmbH/IHP-AnalogAcademy

[34] J. Ratschenberger and H. Pretl, *From code to chip: Open-source automated analog layout design*, 1st ed. in Synthesis lectures on engineering, science, and technology. Springer Cham, 2025. doi: 10.1007/978-3-031-68562-0.

[35] B. Razavi, “The delta-sigma modulator [a circuit for all seasons],” *IEEE Solid-State Circuits Magazine*, vol. 8, no. 2, pp. 10–15, Apr. 2016, doi: 10.1109/MSSC.2016.2543061.

[36] B. Razavi, *Design of analog CMOS integrated circuits*, 2nd ed. McGraw-Hill Education, 2016.

[37] B. Razavi, “The switched-capacitor integrator [a circuit for all seasons],” *IEEE Solid-State Circuits Magazine*, vol. 9, no. 1, pp. 9–11, 2017, doi: 10.1109/MSSC.2016.2624178.

[38] J. M. de la Rosa, “Sigma-delta modulators: Tutorial overview, design guide, and state-of-the-art survey,” *IEEE J. Circuit and Systems*, vol. 58, no. 1, pp. 1–21, Jan. 2011, doi: 10.1109/TCSI.2010.2097652.

[39] J. M. de la Rosa and R. del Rio, *CMOS sigma-delta converters - practical design guide*. John Wiley & Sons, 2013, p. 398. doi: 10.1002/9781118569238.

[40] R. Schreier and G. C. Temes, *Understanding delta-sigma data converters*. Wiley-IEEE Press, 2004, p. 464.

[41] R. Schreier, “Delta sigma toolbox.” MATLAB Central File Exchange, Feb. 2020. Available: https://www.mathworks.com/matlabcentral/fileexchange/19-delta-sigma-toolbox

[42] S. Sutherland and D. Mills, *Verilog and SystemVerilog gotchas*. Springer US, 2007. doi: 10.1007/978-0-387-71715-9.

[43] Z. Tan, C.-H. Chen, Y. Chae, and G. C. Temes, “Incremental Delta-Sigma ADCs: A Tutorial Review,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, pp. 4161–4173, Dec. 2020, doi: 10.1109/TCSI.2020.3033458.

[44] G. C. Temes, U.-K. Moon, and D. Allstot, “Switched-capacitor circuits [education],” *IEEE Circuits and Systems Magazine*, vol. 21, no. 4, pp. 40–42, 2021, doi: 10.1109/mcas.2021.3118195.

[45] T. Verbeure, “An intuitive look at moving average and CIC filters.” GitHub, Sept. 2020. Available: https://tomverbeure.github.io/2020/10/11/Designing-Generic-FIR-Filters-with-pyFDA-and-Numpy.html

[46] T. Verbeure, “PDM microphones and sigma-delta a/d conversion.” GitHub, Oct. 2020. Available: https://tomverbeure.github.io/2020/10/04/PDM-Microphones-and-Sigma-Delta-Conversion.html

[47] T. Verbeure, “Designing generic FIR filters with pyFDA and NumPy.” GitHub, Oct. 2020. Available: https://tomverbeure.github.io/2020/10/11/Designing-Generic-FIR-Filters-with-pyFDA-and-Numpy.html

[48] T. Verbeure, “From microphone datasheet to filter design specification.” GitHub, Oct. 2020. Available: https://tomverbeure.github.io/2020/10/17/From-Microphone-Datasheet-to-Design-Specification.html

[49] T. Verbeure, “Half-band filters, a workhorse of decimation filters.” GitHub, Dec. 2020. Available: https://tomverbeure.github.io/2020/12/15/Half-Band-Filters-A-Workhorse-of-Decimation-Filters.html

[50] M. Wickert, *Signals and systems for dummies*. John Wiley & Sons Inc, 2013. Available: https://learning.oreilly.com/library/view/signals-and-systems/9781118475669/?ar=

[51] A. Williams and F. J. Taylor, *Electronic filter design handbook*, 4th ed. McGraw-Hill Professional, 2006, p. 775.

[52] S. Wolter, “Digital signal processing i.” Course notes reader, Hochschule Bremen (HSB), institution of Electrical Engineering; Computer Science, Bremen, Germany, Dec. 2024. Available: https://wolter.hs-bremen.de/skripte/DSP_I/

[53] S. Wolter, “Digital signal processing II.” Course notes reader, Hochschule Bremen (HSB), institution of Electrical Engineering; Computer Science, Bremen, Germany, Dec. 2024. Available: https://wolter.hs-bremen.de/skripte/DSP_II/

[54] C. Wulff, “Advanced integrated circuits: Lecture notes.” Accessed: Nov. 01, 2026. [Online]. Available: https://analogicus.com/aic2026/

[55] B. V. Zeghbroeck, “Principles of semiconductor devices.” Course notes reader, Univ. of Colorado at Boulder, Dept. of Electrical, Computer; Energy Engineering, Colorado, U.S., 2020. Available: https://truenano.com/PSD20/